```
GET     /widgets           PhoneAppWeb.WidgetController :index
GET     /widgets/:id/edit  PhoneAppWeb.WidgetController :edit
GET     /widgets/new       PhoneAppWeb.WidgetController :new
GET     /widgets/:id       PhoneAppWeb.WidgetController :show
POST    /widgets           PhoneAppWeb.WidgetController :create
PATCH   /widgets/:id       PhoneAppWeb.WidgetController :update
PUT     /widgets/:id       PhoneAppWeb.WidgetController :update
DELETE  /widgets/:id       PhoneAppWeb.WidgetController :delete
```

Notice that some of the routes have :id in them. This is a route parameter that will be passed to the controller action. All of this should feel familiar—routes are defined almost identically to how they are in Rails.

That's enough theory, let's get to code. We'll define our first route, create the controller and action, and render HTML via a template.

Serve Requests with Phoenix Controllers

Phoenix uses the Model-View-Controller (MVC) design pattern. MVC manifests in different ways depending on the framework and use case. MVC has three distinct parts. Models hold and operate on the data of the application. Views represent what the user sees and interacts with. Controllers accept input from a client and coordinate all of the important logic that's needed to serve a request. This all happens in a repetitive cycle as a user uses your application.

Rails uses this same pattern, so this should feel familiar. This pattern emerges naturally as you use Phoenix, so you don't need to worry about the details too much right now. As you proceed through the chapter, you'll see the separation between views, controllers, and your application logic. Let's jump right in and create a controller action.

Write Our First Controller Action

A good strategy to start a new endpoint is to work as the request flows. This means that we'll work from router, to controller, to data, and finally to views. This takes place over several modules across the entire application—everything works together to give us the final result. Add the following routes to the PhoneAppWeb.Router module:

phone_app/lib/phone_app_web/router.ex

```
scope "/", PhoneAppWeb do
  pipe_through :browser

  get "/messages", MessageController, :index
  get "/messages/new", MessageController, :new
  post "/messages/new", MessageController, :create
  get "/messages/:contact_id", MessageController, :show
end
```